



**Instituto Tecnológico de Estudios Superiores de Monterrey,
Campus Querétaro**

TC3002B.201

Desarrollo de aplicaciones avanzadas de ciencias computacionales

Actividad 2 ML: Egg Convolutional Neural Network

Profesor

Benjamín Valdés Aguirre

Presenta

José Diego Llaca Castro

A01704793

Índice

Índice.....	2
Resumen.....	3
Introducción.....	3
Métodos.....	4
Modelo 1.....	4
Modelo 2.....	5
Modelo 3.....	5
Resultados.....	6
Discusión.....	7
Conclusión.....	7
Referencias.....	8

Resumen

Durante este proyecto se realizaron 3 modelos, además de un modelo de línea base para tener una comparación inicial. De los modelos desarrollados únicamente uno tuvo un desempeño aceptable. A lo largo del desarrollo y mejora de los modelos se usaron múltiples herramientas como métricas (F1 score, precisión, recall, etc.), matriz de confusión, balanceo de datos a través de generación de imágenes, entrenamiento a través de generación de imágenes, funciones callback, mejora de imágenes, entre otras. Al usar estas herramientas se logró que el modelo identificará exitosamente múltiples características relevantes para las predicciones realizadas por el modelo, sin embargo todavía hay espacio de mejora especialmente en la identificación de características relevantes para la clasificación de las imágenes (Texturas y tipos de imágenes).

Introducción

En los últimos años ha existido un gran auge de Inteligencias Artificiales de todo tipo, y esto con razón ya que su uso da un apoyo muy versátil para tareas en múltiples áreas tanto de manera profesional como cotidiana. Entre estas herramientas se encuentran las redes convolucionales, las cuales son utilizadas principalmente para el reconocimiento de objetos y clasificación de imágenes.

Con el propósito de comprender su funcionamiento y en base a esta comprensión realizar mejoras en este proyecto se realizó la implementación de una red neuronal convolucional capaz de clasificar imágenes en categorías. Este proyecto originalmente tenía como objetivo ser capaz de distinguir entre hongos y esporocarpos y entre estos dos grupos separar aquellos venenosos de los comestibles, lamentablemente a lo largo del desarrollo del proyecto se volvió evidente que el conjunto de datos seleccionado para el entrenamiento de la red neuronal convolucional no eran aptos para entrenar la red.

Debido a lo anterior se realizó un cambio al alcance del proyecto, ahora apuntando únicamente a la clasificación de hongos venenosos y hongos comestibles. Una vez más tras la revisión del conjunto de datos nuevo se pudieron observar algunos problemas como el desbalance masivo de datos, imágenes corruptas y exceso de ruido, por mencionar algunos. Por lo que nuevamente se realizó un cambio de datos y modificación del alcance, finalmente se llegó a el conjunto de datos seleccionado, el cual contenía aproximadamente 800 imágenes de huevos blancos rotos y enteros.

El último conjunto seleccionado todavía contenía un desbalance de datos, sin embargo este era mucho más fácil de tratar, por ello es que se usó este conjunto y se modificó el alcance del proyecto a la clasificación de huevos rotos y huevos enteros. A través de los diversos cambios del proyecto se usaron tres modelos diferentes, teniendo uno para cada alcance, además de un modelo usado como línea base implementado con EfficientNet para realizar comparaciones con el desempeño de los modelos.

A continuación se hablará sobre estos modelos, su desempeño, los cambios y mejoras realizadas y las razones de estos para así comprender las partes esenciales de su implementación, sus características, y sus posibles mejoras considerando los aprendizajes obtenidos durante el proyecto.

Métodos

Modelo 1

Empezando por la implementación del primer modelo fue muy sencilla ya que al realizarse este modelo los mayores problemas identificados fueron el preprocesado de las imágenes del conjunto de datos. Este conjunto tenía un desbalance en las clases teniendo casi 4 veces más imágenes en una sola categoría comparado con la categoría con menos imágenes del conjunto. Por lo anterior se implementó un código el cual a través de la generación de datos incrementó las imágenes disponibles en cada categoría para evitar el desbalance de datos. En este caso se realizó el aumento de datos debido a que de acuerdo con Buda et al. para conjuntos de datos con 4 categorías y una diferencia de aproximadamente 800 imágenes resulta ligeramente mejor realizar un aumento de datos en vez de la solución alternativa de la eliminación de datos hasta llegar a un balance.

Dejando de lado este problema inicial, para esta etapa no se usó una gran variedad de métricas para comprender el desempeño del modelo, únicamente se tomó en consideración la información proporcionada, siendo esta la matriz de confusión, la exactitud y la pérdida del modelo. Se muestra el resumen del modelo implementado el cual alcanzó una exactitud del 0.25 y una pérdida de 662 aproximadamente.

Model: "sequential_3"		
Layer (type)	Output Shape	Param #
conv2d_8 (Conv2D)	(None, 148, 148, 1)	28
max_pooling2d_5 (MaxPooling2D)	(None, 74, 74, 1)	0
conv2d_9 (Conv2D)	(None, 72, 72, 1)	10
flatten_3 (Flatten)	(None, 5184)	0
dense_12 (Dense)	(None, 1024)	5,309,440
dense_13 (Dense)	(None, 1024)	1,049,600
dense_14 (Dense)	(None, 1024)	1,049,600
dense_15 (Dense)	(None, 4)	4,100
Total params: 7,412,778 (28.28 MB)		

Este modelo no parecía aprender, ya que predecía todas las imágenes dadas como la categoría la cual tenía más imágenes, y al deshacerse del desbalance de datos, lo cual era una explicación lógica para este fenómeno, este seguía prediciendo todas las imágenes en una sola clase. Como se mencionó anteriormente, al explorarse el conjunto de datos se encontraron múltiples anomalías

en las imágenes por lo que se tomó la decisión de cambiar de conjunto de datos.

Al mismo tiempo que se realizó la implementación de este primer modelo también se incluyó el modelo usado como línea base en el cual se incluía en la arquitectura el modelo de EfficientNet teniendo como pesos para este los proporcionados por imagenet. Esta arquitectura fue implementada basándose en el caso de estudio hecho por Kiss et al. Este modelo tampoco tuvo buenos resultados a pesar de tratarse de un modelo pre-entrenado y que se ha demostrado tener un buen desempeño en la clasificación de hongos. Este resultado respaldó la decisión del cambio de datos.

Modelo 2

Después de realizar el cambio de datos gran parte del primer modelo fue conservado, se le hicieron adiciones para aumentar su complejidad esperando obtener mejores resultados. Al realizar el preprocesado de las imágenes nuevamente se descubrieron múltiples problemas, un desbalance masivo de datos teniendo aproximadamente 2 veces más imágenes en una sola categoría comparado contra la suma de imágenes de las tres categorías restantes, debido a limitaciones de espacio y tiempo se tomó la decisión de intentar solucionar este desbalance deshaciéndose de la categoría con más imágenes y la categoría con menos imágenes.

Tras hacer esta reducción se obtuvieron las 2 categorías que tenían aproximadamente la misma cantidad de imágenes, las cuales coincidentemente eran las categorías de hongos comestibles y venenosos. Sin embargo había aún más problemas con los datos, habiendo múltiples imágenes corruptas entre estos, por lo que fue necesario implementar una función la cual se deshiciere de las imágenes inutilizables.

A pesar de todo lo anterior este segundo modelo no presentó ninguna mejora en comparación con el primero, todavía se predecían todas las imágenes como parte de una única categoría. Acto seguido, se realizó una consulta con el profesor Benjamin Valdés, en la que fue señalado que debido al tamaño de las imágenes usadas y la cantidad de “Maxpoolings” en el modelo llegaba un punto en el que era incapaz de extraer características útiles de las imágenes, pues se reducían a pocos píxeles. Al final debido al poco progreso se realizó nuevamente una revisión de las imágenes las cuales volvieron a mostrar que no eran aptas para realizar un buen entrenamiento del modelo, de esta manera también siendo descartadas.

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 150, 150, 32)	896
batch_normalization (BatchNormalization)	(None, 150, 150, 32)	128
max_pooling2d (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_1 (Conv2D)	(None, 75, 75, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 75, 75, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 37, 37, 64)	0
conv2d_2 (Conv2D)	(None, 37, 37, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 37, 37, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 18, 18, 128)	0
conv2d_3 (Conv2D)	(None, 18, 18, 128)	147,584
batch_normalization_3 (BatchNormalization)	(None, 18, 18, 128)	512
max_pooling2d_3 (MaxPooling2D)	(None, 9, 9, 128)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 128)	16,512
dense_1 (Dense)	(None, 4)	516
Total params: 259,268 (1012.77 KB)		

Modelo 3

Finalmente se seleccionó un conjunto de datos mucho más limpio, aunque también mucho más reducido. Este conjunto contenía cerca de 800 imágenes de dos categorías diferentes. Las categorías eran huevos blancos enteros y huevos blancos rotos, lamentablemente hubo nuevamente un desbalance de datos teniendo aproximadamente 600 imágenes en la categoría de huevos enteros y solamente 200 imágenes en la otra categoría. Para solucionar esto se reutilizó el código antes usado para balancear las categorías a través de la generación de imágenes. Teniendo en cuenta los errores cometidos en los modelos

anteriores se disminuyó el uso de capas de Max Pooling ligeramente, sin embargo poco después se volvieron a integrar ya que las nuevas imágenes tenían una mejor resolución por lo que esta vez no había el problema de la reducción excesiva de las imágenes.

A pesar de la mejora del conjunto de datos y del modelo, este todavía seguía clasificando todas las imágenes en la misma categoría. Por ello se realizaron otras estrategias para ayudar al modelo a encontrar patrones en las imágenes más fácilmente. Entre estas acciones fue el preprocesado de las imágenes para aumentar el brillo en imágenes demasiado oscuras y aumentar el contraste permitiendo al modelo observar con mayor facilidad las líneas presentes en las imágenes.

Otra estrategia usada fue el uso de callbacks y pesos, originalmente antes de la reutilización del código se usaron los pesos para evitar tener problemas debido al desbalance de datos, sin embargo al balancear los datos se dejaron de usar los pesos. En cuanto a los callbacks, estos fueron usados para obtener el mejor modelo posible a través del monitoreo de la pérdida del modelo. Una vez que los callbacks implementados (ReduceLROnPlateau, EarlyStopping, ModelCheckpoint) perciben un estancamiento o empeoramiento del modelo se detiene el entrenamiento y se restaura la mejor versión obtenida del modelo.

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 180, 180, 32)	896
conv2d_5 (Conv2D)	(None, 180, 180, 32)	9,248
max_pooling2d_3 (MaxPooling2D)	(None, 90, 90, 32)	0
conv2d_6 (Conv2D)	(None, 90, 90, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 45, 45, 64)	0
conv2d_7 (Conv2D)	(None, 45, 45, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 22, 22, 128)	0
flatten_1 (Flatten)	(None, 61952)	0
dense_2 (Dense)	(None, 128)	7,929,984
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 1)	129
Total params: 8,032,609 (30.64 MB)		

De esta manera se obtuvo un modelo muy similar al inicial, con 2 capas convolucionales (32 nodos cada una) para extraer más fácilmente las características de las imágenes, seguidas de 3 capas de Max Pooling (de 2x2) y 2 capas convolucionales alternadas (64 y 128 respectivamente). Seguidas de estas capas, una capa flatten, una capa densa (128 nodos) y finalmente una capa Dropout (0.35) con el propósito de evitar el overfitting del modelo justo antes de la capa densa final de

un solo nodo para clasificar las imágenes de huevos en sus dos categorías.

Finalmente tras entrenar el modelo y obtener su mejor versión se logró alcanzar una exactitud de 0.8263 en el test, pérdida de 0.3655, precisión de 0.8523, sensibilidad de 0.7895, área bajo la curva de 0.91 y puntaje F1 de 0.8132.

Resultados

El modelo exitoso resultante tiene una exactitud aceptable. Esto se logró tras implementar la generación de imágenes para evitar el desbalance de datos y entrenar el modelo, la integración de métricas relevantes para el diagnóstico y mejoramiento del modelo, permitiendo obtener la mejor versión de este, la purificación del conjunto de datos usados

permitiendo al modelo extraer con mayor facilidad las características relevantes de las imágenes y el uso correcto de técnicas para evitar el overfitting tales como dropout.

Todas estas consideraciones permitieron alcanzar el resultado actual, sin embargo es claro que todavía hay espacio de mejora. A continuación se hablará de las pruebas realizadas con imágenes externas al conjunto de datos.

Discusión

Tras obtener el modelo actual se descargaron 4 imágenes, 2 imágenes de huevos rotos y 2 imágenes de huevos enteros, con el propósito de probar el modelo con imágenes externas. Al realizar las predicciones de estas imágenes 2 fueron clasificadas correctamente mientras la restantes fueron clasificadas incorrectamente. Tras analizar las imágenes con las cuales el modelo fue entrenado y las imágenes externas hay ciertos patrones los cuales podrían ser las causas de las predicciones erróneas.

La primera imagen clasificada erróneamente fue un huevo roto con un fondo blanco. Al comparar esta imagen con otras del conjunto usado para el entrenamiento he observado el patrón que los huevos enteros no muestran ninguna línea dentro del espacio blanco identificado en la imagen, sin embargo en la imagen externa al ser toda completamente blanca el modelo no considera que haya un huevo roto, pues en su gran mayoría el color blanco está agrupado sin líneas que lo separen.

En cuanto a la segunda imagen, se trata de un huevo entero colocado sobre una superficie de color negro y blanco (similar al marmol). En las imágenes usadas para el entrenamiento las imágenes con huevos rotos muestran espacios de color blanco esparcidos en diversos grupos en toda la imagen, por lo que al comparar con la imagen externa es posible que el modelo haya confundido el color del marmol separado por las manchas negras como fragmentos de huevo roto, provocando el error.

Las imágenes que fueron evaluadas correctamente son un huevo roto sobre un plato, permitiendo al modelo identificar sin problema las 2 agrupaciones de color blanco, y un huevo entero en un fondo azul lo que le permite observar sin problemas una sola agrupación de color blanco.

Debido al análisis anterior considero que las principales características que confunden al modelo son las texturas de las imágenes y la curvatura de las líneas. Estos elementos claramente no están siendo observados con suficiente detenimiento por el modelo, ya que si fueran observados no realizaría confusiones en sus predicciones basadas completamente en las agrupaciones de colores. Por ello considero que para mejorar la exactitud del modelo una estrategia a considerar es la integración de modelos de apoyo que identifiquen la presencia de líneas curvas y líneas agudas para usar como factor a considerar al decidir si se tratan de huevos rotos o de simples fondos agrupados de color blanco.

Conclusión

Tras múltiples dificultades, tanto con los conjuntos de datos de entrenamiento usados como con la extracción de características de las imágenes fue posible obtener un modelo

aceptable que clasifica huevos en perfecto estado y huevos rotos. Las principales herramientas que permitieron el desarrollo exitoso de este modelo fueron las múltiples métricas usadas aunadas a la matriz de confusión las cuales permitieron un buen diagnóstico del desempeño del modelo. Además la mejora de calidad y contraste de las imágenes claramente hicieron la diferencia para que el modelo fuera capaz de identificar características relevantes con mayor facilidad.

Al usar estas herramientas junto con la información proporcionada tanto por el profesor Benjamín y las publicaciones en las cuales se basó este proyecto se logró el desarrollo del modelo final el cual a pesar de tener áreas de mejora, tiene una exactitud aceptable con imágenes reales. El próximo paso de este modelo es la implementación de modelos de apoyo que identifiquen la presencia de texturas y tipos de líneas en las imágenes para evitar las confusiones actuales en las predicciones.

Referencias

Buda, M., Maki, A., & Mazurowski, M. A. (2018, August 9). A systematic study of the class imbalance problem in convolutional neural networks. Neural Networks. Advance online publication. <https://doi.org/10.1016/j.neunet.2018.07.011>

IBM. (n.d.). ¿Qué son las redes neuronales convolucionales? IBM Think. <https://www.ibm.com/mx-es/think/topics/convolutional-neural-networks>

Kiss, N., & Czuni, L. (2021, September). Mushroom image classification with CNNs: A case-study of different learning strategies. 2021 12th International Symposium on Image and Signal Processing and Analysis (ISPA). <https://doi.org/10.1109/ISPA52656.2021.9552053>

Ringa Tech (n.d.). Redes Neuronales Convolucionales - Clasificación avanzada de imágenes con IA / ML (CNN). YouTube. Retrieved April 29, 2025, from <https://www.youtube.com/watch?v=4sWhhQwHqug>